

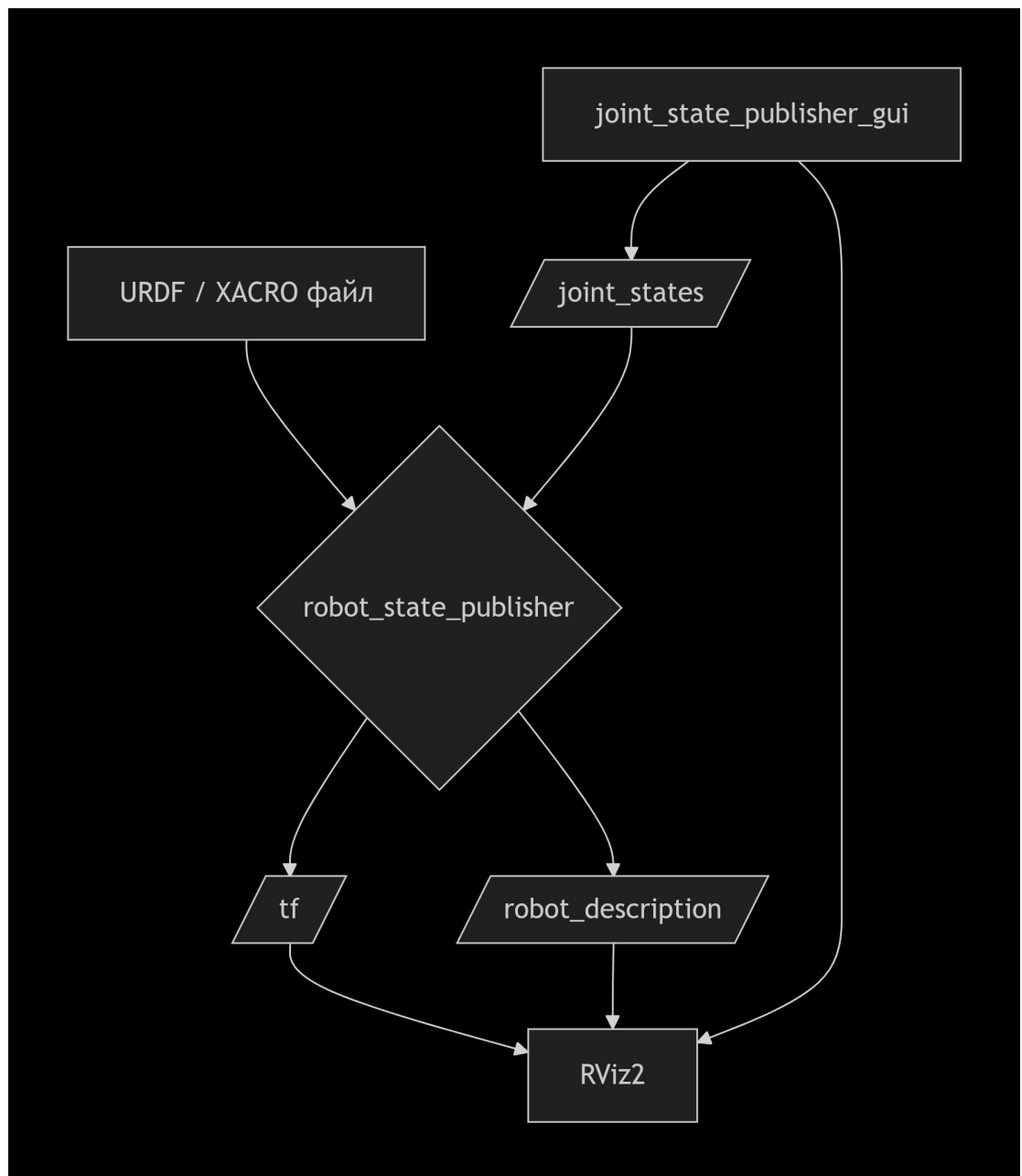
ШПАРГАЛКИ ПО ROS2 ДЛЯ РАЗРАБОТЧИКОВ

TF2, URDF, XACRO И ИНСТРУМЕНТЫ ДЛЯ РАБОТЫ В НИМИ В ROS2

Разработано в РТУ МИРЭА ИИИ КПУ
Грачевым А.В.
Зениной А.А.

ОГЛАВЛЕНИЕ

TF2: Библиотека трансформаций	4
Инструменты для инспекции TF2	4
Ключевая концепция: World Frame	4
URDF: Описание модели робота	5
Структура URDF	5
Инструменты для работы с URDF	5
XACRO: Программирование для URDF	6
Пример: Префиксы	6
Как XACRO превращается в URDF?	6
Экосистема узлов для "оживления" модели	7
1. <i>robot_state_publisher</i>	7
2. <i>joint_state_publisher / joint_state_publisher_gui</i>	7
Сборка всего вместе: Рабочий пайплайн	8



.....	9
Пример Launch-файла (Python)	10
Финальная шпаргалка "В одну строку"	11
Обратная связь	12

TF2: Библиотека трансформаций

tf2 — это библиотека, которая позволяет отслеживать множество систем координат (frames) во времени. Она хранит древовидную структуру связей между ними и даёт возможность преобразовывать вектора, точки и другие данные из одной системы координат в другую в любой момент времени. Если ты когда-либо задавался вопросом: «Где находится лазер дальномёра относительно центра робота?» — ответ даст tf2.

Инструменты для инспекции TF2

Прежде чем погружаться в описание робота, нужно уметь смотреть на то, что получается на выходе.

Команда	Описание	Пример
<code>ros2 run tf2_tools view_frames</code>	Слушает tf2 в течение 5 секунд и генерирует PDF-файл с полным графом всех известных систем координат. Важно: файл создаётся в текущей директории.	<code>ros2 run tf2_tools view_frames</code>
<code>ros2 run tf2_ros tf2_echo <source_frame> <target_frame></code>	Показывает в реальном времени трансформацию между двумя системами координат.	<code>ros2 run tf2_ros tf2_echo base_link laser_frame</code>
rviz2 (с плагином TF)	В Rviz2 нужно добавить дисплей TF. Он покажет все активные системы координат и их связи в 3D-пространстве.	Запустить rviz2, Add -> TF
<code>ros2 run tf2_ros static_transform_publisher</code>	Публикует статическую трансформацию из командной строки (полезно для тестов).	<code>ros2 run tf2_ros static_transform_publisher 0.1 0 0 0 0 0 base_link laser_frame</code>

Ключевая концепция: World Frame

При симуляции или локализации часто возникает путаница: в Rviz2 мы выбираем **Fixed Frame** (обычно `world`, `odom` или `map`). Если робот движется, а в Rviz2 стоит **Fixed Frame = base_link**, то будет казаться, что мир движется вокруг робота. Чтобы увидеть движение робота в пространстве, нужна трансформация от `world` до `base_link`. Эту трансформацию обычно публикует узел локализации (или симулятор, как Gazebo).

URDF: Описание модели робота

URDF (Unified Robot Description Format) — это XML-файл, который описывает кинематику и динамику робота. Он состоит из двух основных элементов: link (звено) и joint (сочленение).

Структура URDF

- > **<link>**: Описывает физическую часть робота. Содержит визуальную (**<visual>**), коллизионную (**<collision>**) и инерциальную (**<inertial>**) информацию.

```
<link name="base_link">
  <visual>
    <geometry>
      <box size="0.2 0.3 0.1"/>
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <material name="blue"/>
  </visual>
</link>
```

- > **<joint>**: Связывает два звена (**parent** и **child**) и определяет их взаимное движение. Самые частые типы: **fixed** (неподвижный), **revolute** (вращательный с ограничениями), **continuous** (бесконечное вращение, как у колеса), **prismatic** (поступательный).

```
<joint name="head_swivel" type="revolute">
  <parent link="base_link"/>
  <child link="head"/>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-3.14" upper="3.14" effort="10"
velocity="1"/>
</joint>
```

Инструменты для работы с URDF

Инструмент	Назначение	Команда
------------	------------	---------

<code>urdf_to_graphiz</code>	Создаёт визуальный граф (в формате PDF) связей между линками и джойнтами из URDF-файла.	<code>urdf_to_graphiz model.urdf</code>
<code>check_urdf</code>	Проверяет синтаксис и структуру URDF-файла на ошибки.	<code>check_urdf model.urdf</code>

ХАСРО: Программирование для URDF

Писать большие URDF-файлы вручную — мучительно. На помощь приходит ХАСРО (XML Macros). Это надстройка над URDF, которая добавляет возможности программирования: макросы, константы, математические выражения и включение файлов .

Фича	Синтаксис	Описание
Константы	<code><xacro:property name="pi" value="3.14159"/></code>	Определяем константы, чтобы не дублировать числа .
Математика	<code> \${pi/2}</code>	Внутри <code> \${}</code> можно писать простые математические выражения.
Включения	<code><xacro:include filename="\$(find my_pkg)/urdf/other.xacro" /></code>	Разбиваешь модель на логические части и собираешь как конструктор .
Макросы	<code><xacro:macro name="wheel" params="prefix side"> ... </xacro:macro></code>	Создаёшь свой элемент, который можно переиспользовать. Например, макрос колеса, который создаёт линк и джойнт для колеса. При вызове <code><xacro:wheel prefix="left" side="1"/></code> генерируется блок кода с уникальными именами .

Пример: Префиксы

Макросы почти всегда принимают параметр `prefix`. Это нужно, чтобы при вызове одного макроса несколько раз (например, для двух колёс) у линков и джойнтов были уникальные имена, и URDF не ругался на дубликаты.

Как ХАСРО превращается в URDF?

Обычно это делает `launch`-файл. Он вызывает программу `xacro`, которая обрабатывает `.xacro` файл и генерирует строку URDF "на лету", чтобы передать её в `robot_state_publisher`.

Экосистема узлов для "оживления" модели

Описание само по себе статично. Чтобы робот двигался в Rviz или симуляции, нужно три вещи:

1. Само описание (`robot_description`).
2. Узел, который умеет читать это описание и вычислять положение всех частей на основе текущего состояния суставов.
3. Источник данных о состоянии суставов.

1. `robot_state_publisher`

Это самый главный узел. Он делает две вещи :

Читает URDF: Он подписывается на параметр `robot_description`, где лежит URDF-модель вашего робота.

Публикует TF2: Он слушает топик `/joint_states` (тип `sensor_msgs/msg/JointState`), в котором публикуются текущие углы/положения суставов. Используя URDF-модель и текущие значения джойнтов, он вычисляет, где находится каждый `link`, и публикует все эти трансформации в `/tf`.

Запуск:

```
# В launch-файле
Node(
  package='robot_state_publisher',
  executable='robot_state_publisher',
  parameters=[{'robot_description': urdf_xacro_string}]
)
```

2. `joint_state_publisher` / `joint_state_publisher_gui`

Эти узлы нужны для тестирования. Они публикуют значения в топик `/joint_states`.

- `joint_state_publisher` (без GUI): Читает из URDF список всех нефиксированных (`non-fixed`) джойнтов и публикует для них значения по умолчанию (обычно 0). Бесполезен для интерактива.
- `joint_state_publisher_gui` : Запускает графическое окошко со слайдерами для каждого джойнта. Двигая слайдеры, вы меняете значения, которые публикуются в `/joint_states`. Идеально для проверки модели, так как вы можете вручную покрутить суставы и увидеть, как движется модель в Rviz.

Установка GUI:

```
sudo apt install ros-<distro>-joint-state-publisher-gui
```

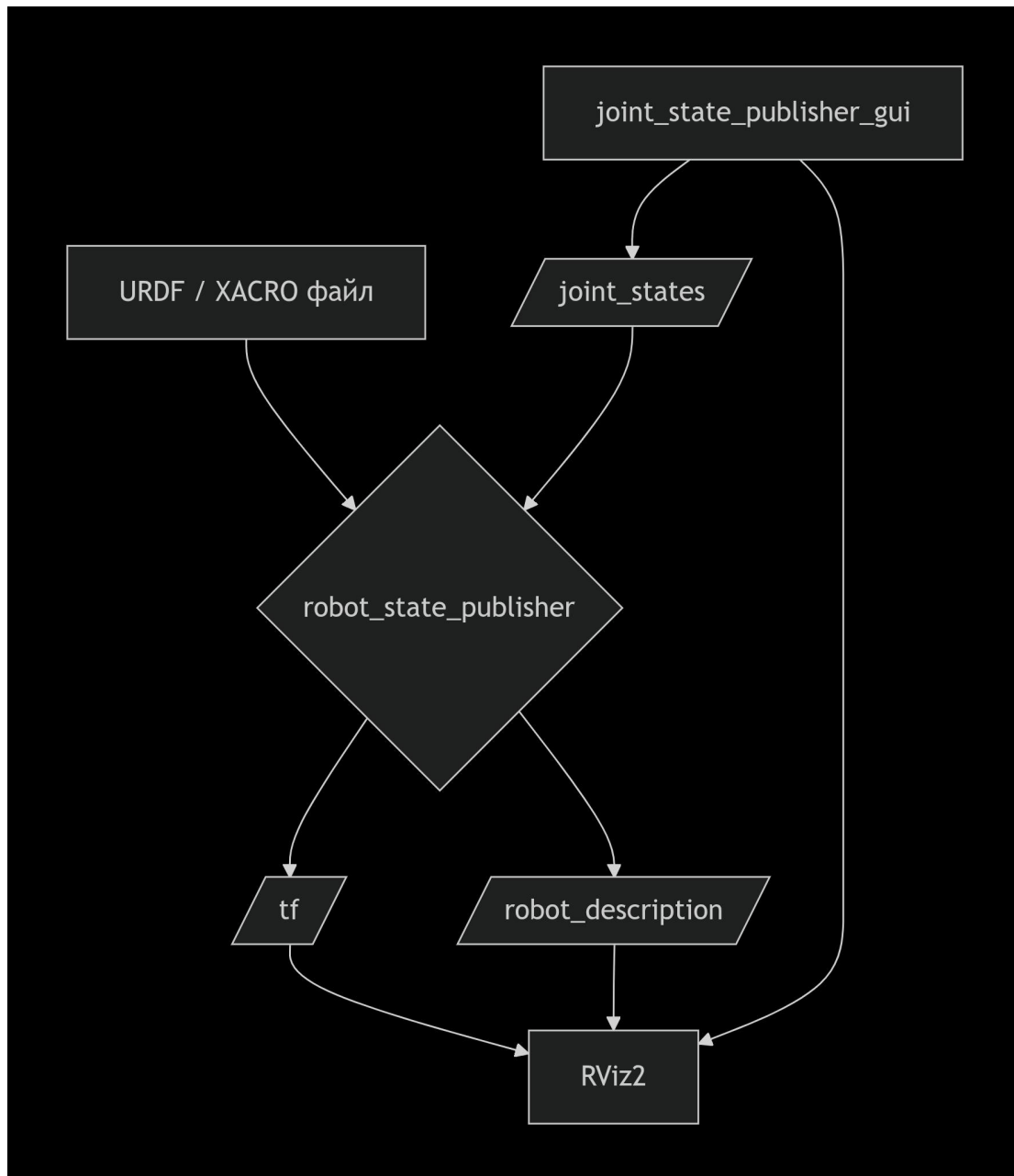
Структура сообщения `/joint_states` (`ros2 topic echo /joint_states --once`):

```
header:
  stamp: ...    # Время
  frame_id: ''  # Обычно пусто
name:          # Массив имён суставов
- shoulder_pan_joint
- shoulder_lift_joint
- ...
position:      # Массив их позиций (углов в радианах)
- 0.0
- -0.5
- ...
velocity: []   # Массив скоростей (опционально)
effort: []     # Массив усилий (опционально)
```

Важно: Порядок имён в массиве `name` должен соответствовать порядку чисел в массиве `position`.

Сборка всего вместе: Рабочий пайплайн

Вот классическая схема, как все эти компоненты взаимодействуют, чтобы вы увидели движущегося робота в Rviz:



1. Источник: У вас есть файл `my_robot.urdf.xacro`.
2. Обработка: Launch-файл читает этот `.xacro` файл с помощью библиотеки `xacro` и превращает его в строку URDF.
3. Запуск: Launch-файл запускает три узла:
 - `robot_state_publisher`, передав ему параметр `robot_description` с той самой URDF-строкой.
 - `joint_state_publisher_gui` (или ваш собственный узел, публикующий `JointState`).
 - `rviz2`.
4. Поток данных:
 - `joint_state_publisher_gui` публикует текущие положения слайдеров в топик `/joint_states`.

- `robot_state_publisher`, имея модель и зная положения суставов из `/joint_states`, вычисляет все трансформации и публикует их в `/tf`.
- `Rviz2` подписывается на `/tf` и на параметр `/robot_description`. Зная, где находится каждый линк и как он выглядит, `Rviz2` отрисовывает робота.

Пример Launch-файла (Python)

Этот код делает именно то, что описано выше: обрабатывает `.xacro` и запускает всё необходимое .

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
import xacro

def generate_launch_description():
    # Путь к XACRO файлу
    xacro_file = os.path.join(
        get_package_share_directory('my_package'),
        'urdf', 'my_robot.urdf.xacro')

    # Генерация URDF строки с помощью xacro
    robot_desc = xacro.process_file(xacro_file).toxml()

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            parameters=[{'robot_description': robot_desc}]
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
            name='joint_state_publisher_gui'
        )
    ])
```

```

Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2'
)
])

```

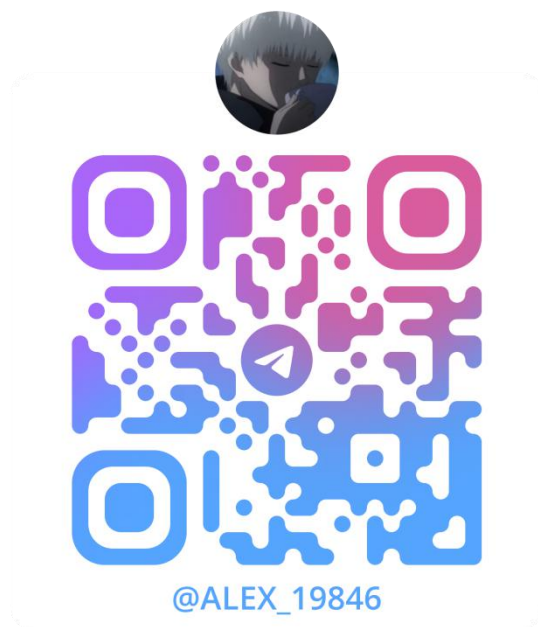
Финальная шпаргалка "В одну строку"

Ты хочешь...	Тебе нужно...	Команда/Инструмент
Увидеть дерево систем координат	<code>tf2_tools</code>	<code>ros2 run tf2_tools view_frames</code>
Проверить трансформацию между двумя звеньями	<code>tf2_echo</code>	<code>ros2 run tf2_ros tf2_echo base_link head_link</code>
Проверить URDF на ошибки	<code>check_urdf</code>	<code>check_urdf model.urdf</code>
Визуализировать URDF как граф	<code>urdf_to_graphviz</code>	<code>urdf_to_graphviz model.urdf</code>
Упростить описание робота с макросами и математикой	XACRO	Использовать расширение <code>.urdf.xacro</code>
Включить один XACRO-файл в другой	<code><xacro:include></code>	<code><xacro:include filename="\$(find pkg)/other.xacro"/></code>
Загрузить модель и транслировать TF2	<code>robot_state_publisher</code>	Запустить ноду с параметром <code>robot_description</code>
Вручную покрутить суставы для теста	<code>joint_state_publisher_gui</code>	Запустить ноду <code>joint_state_publisher_gui</code>
Посмотреть на робота в 3D	<code>rviz2</code>	Запустить, добавить дисплей <code>RobotModel</code> , выбрать <code>Fixed Frame</code>

Обратная связь

Если Вы нашли ошибку, неточность, у Вас есть предложения по улучшению или вопросы, напишите в Telegram Александру (https://t.me/Alex_19846) или Алисе (https://t.me/Kika_01). QR-коды со ссылками на их аккаунты представлены ниже.

Александр



Алиса

